

UniCon: 범용 클러스터의 연결 요소 계산

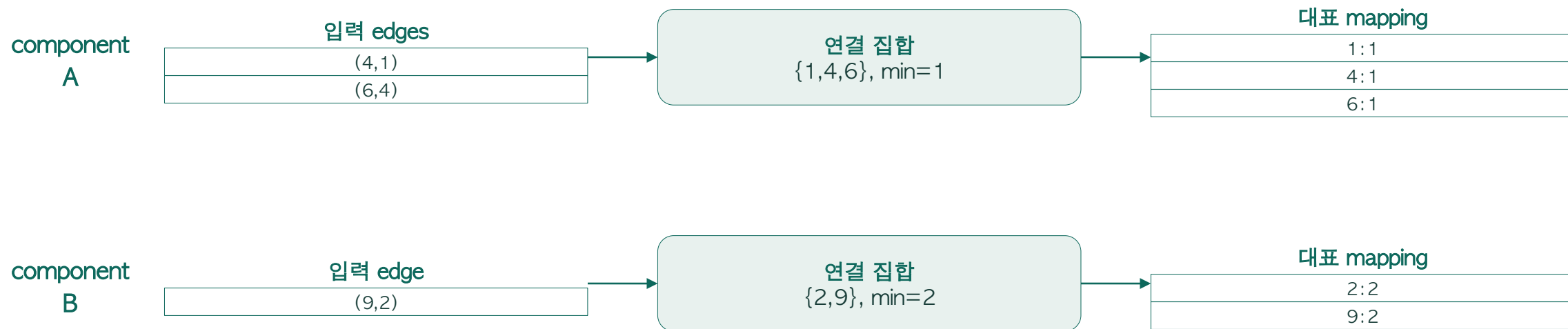
UniCon: A Unified Star-Operation to Efficiently Find Connected Components on a Cluster of Commodity Hardware

Chae-eun Kim, Changhun Han, and Ha-Myung Park (2022)

Unified star, overlap partition, edge filtering, HybridMap을 하나의 정확한 흐름으로 읽는다

Connected components의 정확한 출력

각 vertex u 를 같은 component에서 ID가 가장 작은 representative로 mapping한다



UniCon의 representative는 임의의 label이 아니라 component의 minimum vertex ID다

기호와 상태의 의미

Formula보다 먼저 graph, partition, threshold, 보관 edge를 정의한다

기호	정의	발표에서의 역할
$G=(V,E)$	Vertex set V 와 undirected edge set E	현재 graph
ρ	Partition 개수	Worker-local view 수
$\xi(u)$	Vertex u 의 random hash partition ID	Component label이 아님
τ	Single-machine Rem 전환 threshold	Hardware별 tuning
F	case 1/2의 sep edge 누적 set	Finishing에서 복원

Partition ID와 component representative는 서로 다른 label이다

대규모 CC에서 memory와 I/O가 함께 문제다

Input graph뿐만 아니라 반복 중 생성되는 records도 저장하고 이동해야 한다

방식	주 저장 위치	Scale 한계	남는 비용
Single machine	한 machine의 RAM	RAM 초과 시 OOM	외부 저장 algorithm 필요
Distributed memory	여러 worker의 RAM	전체 graph와 state가 RAM 초과	Machine 수와 memory 의존
MapReduce	Distributed storage	RAM 한계 완화	Disk, sort, network I/O
UniCon 목표	Storage와 local RAM 혼합	Intermediate records 억제	I/O 자체는 남음

Distributed storage를 쓰면 OOM 위험은 줄지만 intermediate-data I/O가 병목이 된다

Alternating, PACC, UniCon의 차이

세 방법은 같은 star operation을 다른 이름으로 부르는 관계가 아니다

Method	반복 operation	Partition awareness	핵심 한계 또는 변화
Alternating	Large-star와 small-star 교대	없음	두 distributed rounds 반복
PACC	PA-large-star와 PA-small-star 교대	있음	Load balance 개선, data 이동 지속
UniCon	UniStar 하나를 반복	Overlap subgraph	Local CC와 중복 억제 결합
DSP-CC	External-memory Union-Find 계열	해당 없음	별도 single-machine method

UniCon은 PACC sketching을 쓰지만 partitioning의 star semantics를 새로 정의한다

UniStar-naive의 data explosion

같은 edge를 양 endpoint 관점에서 다시 만들면 다음 round에서 그 copies가 또 복제된다

01 Original $|E|=14$



02 Round 1 $|E|=25$



03 Round 2 $|E|=29$

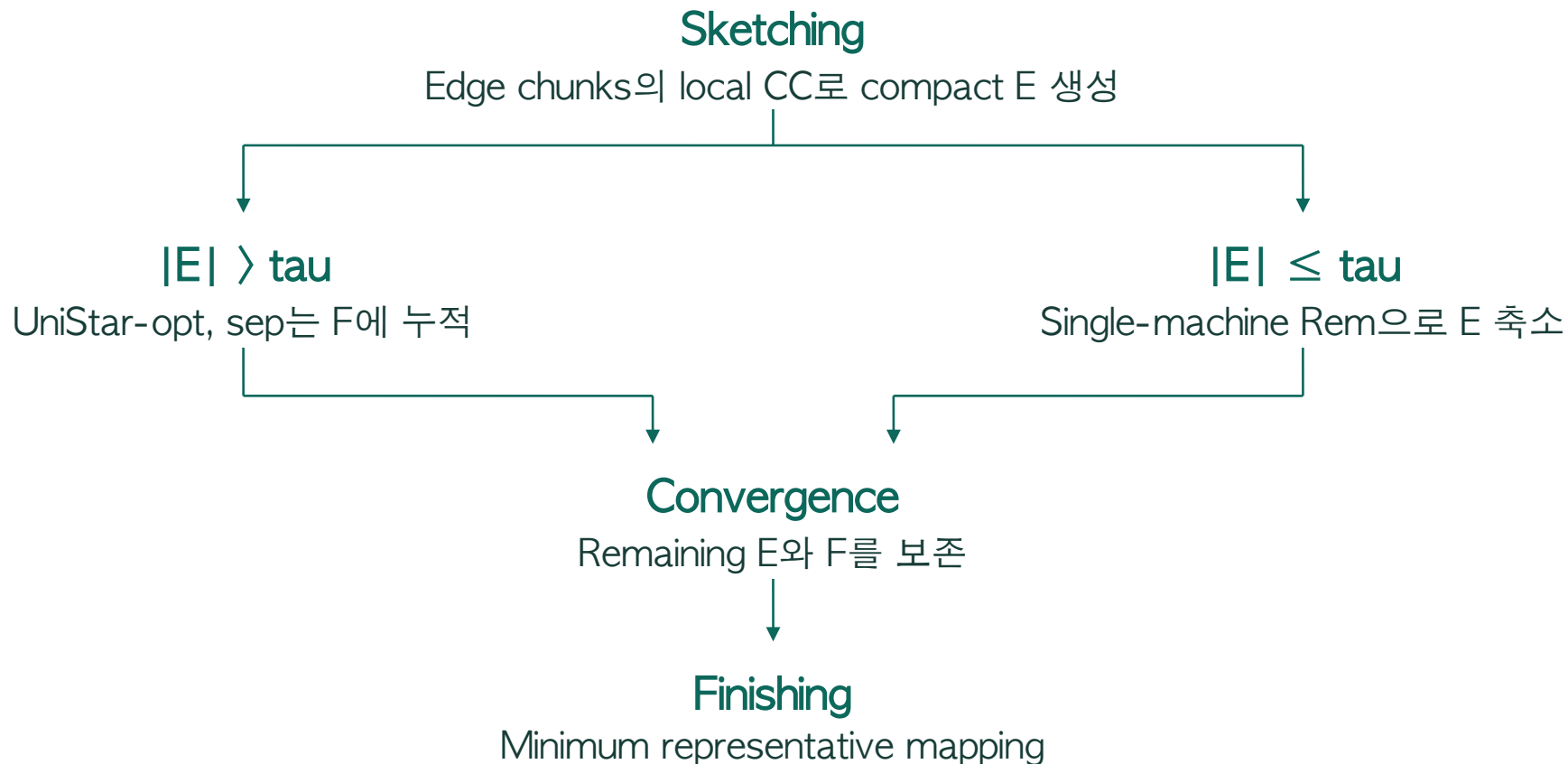


04 Repeated copies Edge (17,16)이 (17,12)와 (17,16)으로 재발행

Unified operation 자체보다 duplicate propagation을 막는 semantics가 중요하다

UniCon의 전체 pipeline

Sketching 뒤의 edge count가 threshold τ 보다 큰지에 따라 distributed round와 Rem이 갈린다



Partition은 disjoint output group이 아니다

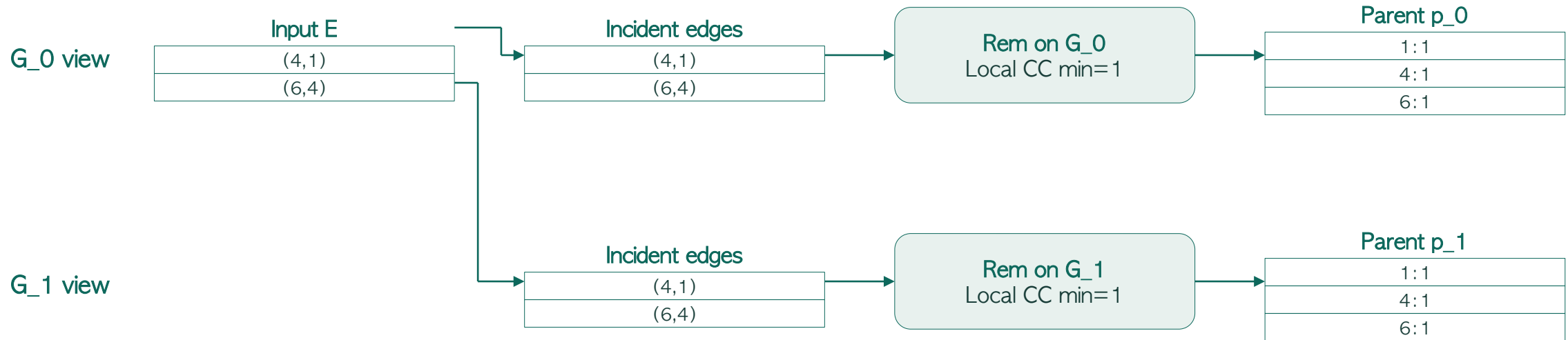
G_i 는 partition i 의 vertices에 incident한 edges로 만든 local view다

- $x_i: V \rightarrow [\rho]$ 는 random hash이며 vertex마다 work partition을 지정한다.
- Partition i 의 edge set E_i 는 $[V]_i$ vertices에 incident한 edges를 포함한다.
- Cross-partition edge는 endpoint가 속한 두 subgraph에 모두 나타날 수 있다.
- 따라서 $G_0, G_1, \dots$ 은 서로 겹치는 subgraphs이며 component partition이 아니다.

Overlap은 중복 비용의 원인이면서 cross-partition connectivity를 보존하는 재료다

작은 path에서 overlap과 Rem 따라가기

입력 $E = \{(4,1), (6,4)\}$, $\rho = 2$, $x_i(1) = 0$, $x_i(4) = 1$, $x_i(6) = 0$ 인 교육용 계산



같은 cross edges가 두 local views에 존재하며 각 Rem은 같은 minimum parent를 찾는다

Parent mapping에서 output edge까지

두 local outputs의 set union은 duplicate를 한 번만 남기고 같은 component를 표현한다

State	Vertex 1	Vertex 4	Vertex 6	Edge set
Input	self	adjacent to 1,6	adjacent to 4	$\{(4,1),(6,4)\}$
p on G_0	1	1	1	local CC min=1
p on G_1	1	1	1	local CC min=1
Union output	representative	4 to 1	6 to 1	$\{(4,1),(6,1)\}$

Input path와 output star는 edge가 달라도 component {1,4,6}을 동일하게 보존한다

UniStar 한 round의 실제 계산

Partition-local CC를 먼저 찾고 같은 hash group의 local minimum을 경유해 output edges를 만든다

01	Hash vertices	$x_i(u)$ 로 ρ partitions 지정
↓		
02	Build G_i	$\{V\}_i$ 에 incident한 edges를 모아 overlap view 생성
↓		
03	Run Rem	$p(u)$ =minimum of local component 계산
↓		
04	Emit local stars	Partition-local minimum 또는 $p(u)$ 에 연결
↓		
05	Set union	모든 G'_i edges를 합치고 duplicate 제거

Global minimum 대신 local minimum 경로로 connectivity를 유지한다

Intact tag가 duplicate input을 줄인다

Intact는 한 partition output에서 edge가 변하지 않았다는 상태 tag다

현재 edge 상태	Map emission	다음 round 의미
Cross edge, tag 없음	$x_i(u)$ 와 $x_i(v)$ 에 모두 emit	두 overlap views에 포함
Partition i에서 intact tag	반대쪽 duplicate emission 생략	한 local view에서 유지
Reduce 시작	이전 tags를 clear	새 output 변화로 tag 재판정
Connectivity	Edge 자체를 모두 제거하지 않음	중복 record만 제외

Intact exclusion은 이미 안정된 cross edge의 두 번째 copy를 생략하는 규칙이다

UniStar-opt의 세 filtering cases

Case 1과 2는 sep에 보관하고 case 3만 counterpart가 남는 조건에서 discard한다

Case	판정 핵심	Next round	Finishing
1	Same partition, u 의 following neighbor 없음	제외	sep to F로 복원
2	$xi(v)=i$, v 의 모든 incident edge가 intact	제외	sep to F로 복원
3	$xi(u)=i$, u 의 유일 incident edge이며 intact	Discard	복원 불필요

Filtering은 세 edge를 같은 방식으로 삭제하는 optimization이 아니다

Filtered edge의 state 변화

다음 round input에서 빠진다는 사실과 final result에서 사라진다는 사실은 다르다

01	UniStar-opt	Output edge의 filtering case 판정
↓		
02	Case 1 or 2	Edge를 sep로 emit
↓		
03	Accumulate F	여러 rounds의 sep를 set union
↓		
04	Rem on remaining E	$ E $ 가 τ 이하이면 single-machine Rem
↓		
05	Finishing	Remaining E와 F로 final mapping 계산

F는 버린 edge 모음이 아니라 final connectivity를 위해 보존한 evidence다

Case 3 discard의 보존 조건

$x_i(u)=i$ 에서 u 의 유일 incident edge가 intact이면 반대 partition의 copy 또는 대체 edge가 남는다

- Original non-intact cross edge는 처음에 두 endpoint partitions에 존재한다.
- 한 view에서 intact가 되어 제외되더라도 opposite view가 같은 edge를 유지할 수 있다.
- Opposite view가 edge를 바꾸면 u 와 다른 neighbor를 잇는 replacement가 생긴다.
- 따라서 case 3의 local duplicate는 final connectivity를 끊지 않는다.

Case 3은 arbitrary deletion이 아니라 counterpart 또는 replacement existence에 의존한다

Threshold tau와 Rem 전환

Tau는 correctness constant가 아니라 distributed I/O와 single-machine memory 사이의 tuning parameter다

Condition	실행	이유	위험
$ E > \tau$	UniStar-opt	Distributed reduction 지속	Round I/O
$ E \leq \tau$	Rem(E)	추가 distributed rounds 절약	Single-machine OOM
$\tau=2000M$	GSH, CW, HL에서 실패	Too-early Rem 전환	한 machine memory 초과
Reported result	Dataset별 optimal tau	Measured tuning	Universal default 아님

Tau를 크게 잡으면 rounds는 줄 수 있지만 remaining graph가 한 machine에 맞지 않을 수 있다

HybridMap이 worker RAM을 구성하는 방법

자기 partition vertices는 array에, 외부 incident vertices는 hash table에 둔다

Node group	Data structure	Access 특성	Memory role
$\{V\}_i$	Compact array	Index access가 빠름	Expected $ V /\rho$
V_i minus $\{V\}_i$	Hash table	외부 key만 저장	At most incident-edge scale
Original Rem p	Universe-size array	빠른 index	Worker마다 $ V $ 라 OOM 위험
All-hash alternative	Hash table	$O(V_i)$ space	Array보다 lookup 느림

HybridMap은 full array를 피하고 frequent local keys에 array speed를 쓴다

Theorem 3의 expected memory bound

V는 vertices, E는 edges, rho는 partitions이며 bound는 random and even hash assumption에 조건부다

Term	Expected worker share	근거	보장하지 않는 것
Local array	$ V /\rho$	Random vertex partition	Worst-case exact size
Incident edges	$2 E /\rho$	Cross edge가 두 views에 가능	Tail concentration
HybridMap total	$O((V + E)/\rho)$	Array plus external keys	Fixed byte ceiling
Increasing rho	Expected memory 감소	더 작은 local views	Runtime always 감소

Theorem 3은 expected asymptotic bound이며 16GB worker의 worst-case 안전 증명은 아니다

증명과 측정을 구분하는 기준

Connectivity theorem, expected memory theorem, empirical speedups는 서로 다른 종류의 claim 이다

Claim	종류	Condition	범위
G와 G'의 same connectivity	Theorem 1	UniStar construction	Correctness
Case 3 counterpart remains	Theorem 2	Cross edge와 only-edge condition	Safe discard
Expected memory bound	Theorem 3	Random and even hash	Asymptotic expectation
4x, 8x, 4096x	Experiment	2022 stack and datasets	Measured comparison

실험 배수는 theorem constant가 아니며 record 감소와 byte 감소도 다르다

Experiment hardware와 software 조건

결과는 10대 commodity machines와 2022년 공개 구현 조건에서 측정됐다

Item	Reported condition	해석 경계
Cluster	10 machines, master also worker	All 10 workers used by default
Each machine	Xeon E3-1220, 4 cores, 3.10GHz	CPU generation specific
Memory and storage	16GB RAM, two 1TB SSDs	Commodity condition
Stack	Hadoop 3.2.1, Spark 3.0.1, MPICH 3.3	Stack-specific
Rem	C++11 implementation	Single-machine phase

성능 배수는 hardware, implementation, tau와 rho tuning을 함께 적어야 한다

UniStar와 naive 통합의 비교

UniCon-base와 UniCon-naive를 비교해 partition-aware processing과 intact exclusion 효과를 측정했다

Metric	Reported maximum	Comparison	Source condition
Round input ratio	naive / UniStar up to 4x	Fig. 8 datasets	Input records
Map output ratio	naive / UniStar up to 8x	Fig. 8 datasets	Intermediate records
Operation time ratio	naive / UniStar = 2 to 5x	Every reported case	Fig. 9
Total time ratio	naive / base up to 4.3x	Rounds also fewer	Fig. 9

UniStar의 효과는 data records 감소와 fewer rounds가 함께 만든 조건부 결과다

Filtering과 HybridMap의 measured effects

두 수치는 CW graph와 해당 experiment configuration에 묶여 있다

Technique	Reported result	Condition	해석
Edge filtering	80.4% average input shrink per round	CW, $\tau=0$	Case 1/2 edges return at finishing
HybridMap	22.7% lower runtime	CW, vs all-hash	Array access benefit
Array-only p	OOM on GSH, CW, HL	$ V $ array per worker	Universe array bottleneck
All-hash p	Memory succeeds	Hash lookup	HybridMap보다 느림 on CW

80.4%와 22.7%는 universal constants가 아니라 CW에서의 measured results다

Scale 결과의 조건과 범위

4096x는 이 cluster에서 LACC와 FastSV가 먼저 OOM한 RMAT comparison이다

Result	Reported value	Exact context	일반화 한계
Largest relative scale	4096x larger	Vs LACC and FastSV on RMAT	Hardware-independent law 아님
Largest real graph HL	3.443B vertices, 128.737B edges	Table 2	Prepared CC input
CW success	Only UniCon and PACC	10-machine experiment	다른 implementation 조건
PACC comparison	PACC time / UniCon-opt time = 1.43	Real-world set report	143% faster로 쓰지 않음

4096x는 graph size의 실험 비교이며 runtime 배수나 theorem bound가 아니다

UniCon 결과의 적용 한계

Correctness와 measured scalability를 다른 graph task나 system stack으로 자동 확장하지 않는다

- Theorem 3은 expectation이며 reducer별 worst-case RAM 또는 tail bound를 주지 않는다.
- Record count 감소는 compressed bytes, sort spill, network traffic의 같은 비율 감소를 보장하지 않는다.
- Tau와 rho는 hardware와 dataset에 따라 조정해야 한다.
- 2022년 CC 결과를 triangle, shortest path, arbitrary graph processing에 적용할 근거는 없다.

UniCon은 CC 전용이며 system condition이 바뀌면 성능을 다시 측정해야 한다

UniCon의 기여를 이루는 네 요소

한 요소만 떼어 설명하면 correctness, I/O reduction, worker memory의 연결이 사라진다

01 Unified star Overlapping G_i에서 local CC 계산



02 Intact exclusion Stable cross-edge duplicate를 다음 round에서 생략



03 Recoverable filtering Case 1/2는 F에 보관, case 3만 discard



04 HybridMap Local array와 external-key hash를 결합

UniCon은 unified star, duplicate suppression, edge preservation, HybridMap의 결합이다

정확한 UniCon 설명

Output, overlap, filtering, evidence boundary를 한 문장씩 확인한다

정확한 설명

CC output은 vertex별 minimum representative mapping이다

Cross edge는 overlapping G_i views에 있을 수 있다

Case 1/2 edges는 F를 거쳐 finishing에 돌아온다

4096x는 10-machine RMAT OOM comparison이다

피해야 할 오해

Partition ID가 component ID다

각 edge는 정확히 한 worker만 본다

Filtering은 모두 영구 삭제다

모든 hardware에서 4096배 확장된다

핵심은 intermediate records를 줄이며 connectivity evidence를 보존하는 방법이다
